

Semantic Comparison of LTL Specifications with NuSMV

Formal Methods for Software Engineering WS22/23

First Author and Second Author

Bauhaus-Universität Weimar

Abstract. Model checking is an automatic technique for verifying correctness properties of safety-critical reactive systems. LTL is a popular logic for the specification of correctness properties. NuSMV is a classic model checking tool supporting the analysis of transition systems and LTL specifications.

We develop a methodology and tools for semantically comparing NuSMV specifications. The input of our tool are two LTL specification and a model. Our tool checks refinement, equivalence, or incomparability of the provided specifications.

1 Introduction

1.1 Background

Model checking is an automatic technique for verifying correctness properties of safety-critical reactive systems [1]. Examples include ...

LTL is a popular logic for the specification of correctness properties. The main operators of LTL are ...

NuSMV¹ is a classic model checking tool supporting the analysis of transition systems and LTL specifications. It is used either for the direct analysis of models and specifications or programmatically by other tools. e.g., Alloy².

1.2 Motivation

When developing safety critical or high-assurance systems LTL specifications become artifacts of the software engineering life-cycle. The specifications have to be written, maintained, updated, refactored etc. as the software system is created and evolved. Writing and reading LTL specifications is a non-trivial task. To date there are no tools that support the semantic comparison of LTL specifications as a system evolves. We aim to develop a methodology and tools for semantically comparing NuSMV specifications to support software engineers writing and maintaining LTL specifications.

¹ Project page of NuSMV at <https://nusmv.fbk.eu/>

² For Alloy 6, see <https://alloytools.org/>

2 Problem Statement

The input of our tool are a model $\mathcal{M}$ and two LTL properties ϕ and ψ .

The possible outputs of a semantic comparison are:

ϕ **refines** ψ iff $\forall \pi \in \mathcal{M}.(s \models \phi) \rightarrow (s \models \psi)$ denoted $\phi \prec \psi$

ψ **refines** ϕ iff $\forall \pi \in \mathcal{M}.(s \models \psi) \rightarrow (s \models \phi)$ denoted $\psi \prec \phi$

equivalent iff $\phi \prec \psi$ and $\psi \prec \phi$ denoted $\phi \equiv \psi$

incomparable otherwise

Optionally, we want the tool to ignore the model $\mathcal{M}$ and perform the semantic comparison for all possible models.

3 Example & Intuition

Consider the example model of a simple request answering system. The engineers initially formulated the LTL specification `eventuallyBusy` in lines 12-13 stating that incoming requests will eventually be worked on in state `busy`. After carefully checking the implementation the team wants to strengthen the specification to require that the `busy` state is reached immediately or in the next step (see lines 15-16).

```

1 MODULE main
2 VAR
3   request : boolean;
4   state   : {ready, busy};
5 ASSIGN
6   init(state) := ready;
7   next(state) := case
8     state = ready & request : busy;
9     TRUE                     : {ready, busy};
10  esac;
11
12 LTLSPEC NAME eventuallyBusy :=
13   G(request -> F state = busy)
14
15 LTLSPEC NAME nowOrNextBusy :=
16   G(request -> (state = busy | X state = busy))

```

Listing 1.1. NuSMV model example with two named LTL specifications to compare

Our tool enables the team to check whether this change was indeed a refinement. Intuitively, we generate new LTL specifications checking the comparison via NuSMV. The checks for all properties are shown in Listing 1.2.

```

1 LTLSPEC -- eventuallyBusy refines nowOrNextBusy
2   (G(request -> F state = busy)) ->
3   (G(request -> (state = busy | X state = busy)))
4
5 LTLSPEC -- nowOrNextBusy refines eventuallyBusy

```

```

6   (G(request -> (state = busy | X state = busy))) ->
7   (G(request -> F state = busy))

```

Listing 1.2. Checks for the semantic comparison of LTL specs `eventuallyBusy` and `nowOrNextBusy`

This translation takes ...

4 Implementation

We have written our tool in Java. No external libraries were used, but a binary of the NuSMV model checker is required (and provided as part of our code). Our implementation consists of 800 lines of code in 5 different classes. An overview of the classes is given below ...

4.1 Code Example

We provide and describe two code examples here.

Some code is shown in Listing. 1.3.

```

1  @Test
2  public void
3      givenRegex_whenMatchesWithMultilineFlag_thenCorrect() {
4      int matches = runTest(
5          "dog$", "This is a dog" + System.getProperty("line.
6          separator")
7          + "this is a fox", Pattern.MULTILINE);
8      assertTrue(matches > 0);
9  }

```

Listing 1.3. RegEx example from the web

<https://www.baeldung.com/regular-expressions-java>

4.2 Tool usage

Our tool requires Java 11 or later to run. We have included the binaries of NuSMV for Windows 64Bit (x86) and Linux 64Bit (x86). For other platforms, compile and provide the binaries in the subfolder `./nusmv/bin`.

We implemented a commandline interface with the following options:

- `$filename` (mandatory) the name of the file
- `$spec1` (mandatory) the name of the first LTLSPEC to check
- `$spec2` (mandatory) the name of the second LTLSPEC to check
- `noModel` (optional) if present the model will be ignored
- `showExample` (optional) if present the tool will generate counterexamples (for non-refinement, i.e., two if incomparable)

An example execution for the file shown in Listing 1.1 would look like this:

```
java -jar ltlcompare.jar arbiter.smv eventuallyBusy nowOrNextBusy
```

The tool prints an output to the console. The output contains ...

4.3 Limitations of the Implementation

Our implementation uses regular expressions to find LTLSPEC blocks within a NuSMV file. In cases where the LTL formula is not terminated with a semicolon or where it contains comments that include a semicolon our approach might not capture the correct formulas. Right now there is no easy mitigation of this limitation other than making sure about the above. An alternative implementation should have been based on a NuSMV parser instead of using regular expressions.

When checking semantic properties of specifications without considering the model our implementation attempts to comment out **ASSIGN**, **TRANS**, **INIT**, and **INVAR** blocks. Again we identify these blocks using regular expressions, which is prone to the same errors mentioned above. We have decided to follow this approach rather than extracting the variables as the NuSMV files might declare and compose multiple modules whose structure must be preserved. An alternative implementation should have been based on a NuSMV parser instead of using regular expressions.

5 Evaluation

5.1 Validation

To ensure the correctness our implementation we have tested our prototype on many NuSMV files. We have done a semantic comparison of each LTL property ψ against the special properties:

- ψ : must yield equivalence
- **TRUE**: must yield ψ **refines** **TRUE**
- **FALSE**: must yield **FALSE** **refines** ψ
- $\neg\psi$: must yield **incomparable**

These checks are automated in `AllPropCheckTest.java`. We have inspected all failures and concluded that these result only from ψ being either trivially **FALSE** or trivially **TRUE**.

We have further checked special cases where we manually designed pairs of specifications to have fixed comparison outcomes. These checks include 17 LTL specification pairs on 9 models and are automated in `SpecialCasesTest.java`.

5.2 Performance Cost

We have analysed the performance cost of the semantic comparison. Our corpus of specifications consists of 30 NuSMV files obtained from public GitHub repositories³. We have measured the times of the validation checks described above. All experiments were executed on a laptop computer with an i7 CPU at 2.4GHz.

We have computed the average analysis times as 900ms with maximum times at 10sec for specification

³ We have used the search query <https://github.com/search?l=&q=module+ltlspec+extension%3Aasmv&ref=advsearch&type=Code>

6 Discussion & Conclusion

Eventually, we did a great job in this project and would like to do some extension of it in our MSc thesis.

References

1. Clarke, E.M., Schlingloff, B.: Model checking. In: Handbook of Automated Reasoning, pp. 1635–1790. Elsevier and MIT Press (2001)